

```
In [14]: import os
import time
from pathlib import Path

import numpy as np
import pandas as pd
import requests

os.environ["CENSUS_API_KEY"] = "b1278b466e026ad679a62a03dd4b044bb8b36e7c"
CENSUS_API_KEY = os.getenv("CENSUS_API_KEY")
print("Key loaded:", CENSUS_API_KEY is not None)

pd.set_option("display.max_columns", 200)
pd.set_option("display.width", 200)
pd.set_option("display.max_colwidth", 120)

# -----
# Paths / config
# -----
DATA_DIR = Path("data")
RAW_DIR = DATA_DIR / "raw"
OUT_DIR = DATA_DIR / "processed"
RAW_DIR.mkdir(parents=True, exist_ok=True)
OUT_DIR.mkdir(parents=True, exist_ok=True)

# Update this path if needed
IGS_PATH = r"C:\Users\jabba\Desktop\Code\machine_learning\AUC_mastercard_cha

# Tract selection for focused EDA / testing
TARGET_TRACTS = None
# Example:
# TARGET_TRACTS = ["13089023301", "13089023405", "13089023302"]

# If TARGET_TRACTS is set, the code will infer state FIPS from those tracts
SAVE_INTERMEDIATE = True

# ACS 5-year currently available through 2024
ACS_MAX_AVAILABLE_YEAR = 2024
FILL_2025_WITH_2024_ACS = False # optional, keep False by default
```

Key loaded: True

Cell 2 — IGS load / clean helpers

```
In [15]: IGS_STRING_COLS = {
    "County",
    "State",
    "BENCHMARK",
    "Census Tract FIPS code",
    "geoid",
}

IGS_ALIAS_MAP = {
    "Inclusive Growth Score": "igs_total",
```

```

    "Growth": "igs_growth",
    "Inclusion": "igs_inclusion",
    "Place": "igs_place",
    "Economy": "igs_economy",
    "Community": "igs_community",
    "Place Growth": "igs_place_growth",
    "Place Inclusion": "igs_place_inclusion",
    "Economy Growth": "igs_economy_growth",
    "Economy Inclusion": "igs_economy_inclusion",
    "Community Growth": "igs_community_growth",
    "Community Inclusion": "igs_community_inclusion",
}

def dedupe_columns(cols):
    seen = {}
    out = []
    for c in cols:
        c = str(c).strip()
        if c not in seen:
            seen[c] = 0
            out.append(c)
        else:
            seen[c] += 1
            out.append(f"{c}.{seen[c]}")
    return out

def _find_header_row(df_raw, required_tokens=("Census Tract FIPS", "Year")):
    tokens = [t.lower() for t in required_tokens]
    for i in range(min(len(df_raw), 50)):
        row = df_raw.iloc[i].astype(str).str.lower().tolist()
        if all(any(tok in cell for cell in row) for tok in tokens):
            return i
    raise ValueError("Could not find the real IGS header row.")

def normalize_geoid_series(series: pd.Series) -> pd.Series:
    s = series.astype("string").str.strip()

    numeric = pd.to_numeric(s, errors="coerce")
    numeric_mask = numeric.notna()

    s = s.copy()
    s.loc[numeric_mask] = numeric.loc[numeric_mask].astype("Int64").astype("string")

    s = s.str.replace(r"\.0$", "", regex=True)
    s = s.str.replace(r"\D", "", regex=True)
    s = s.str.zfill(11)
    s = s.where(s.str.fullmatch(r"\d{11}"), pd.NA)

    return s

def filter_tracts(df: pd.DataFrame, tracts=None, geoid_col="geoid") -> pd.DataFrame:
    if not tracts:

```

```

        return df.copy()

    tract_set = {str(t).zfill(11) for t in tracts}
    out = df[df[geoid_col].astype("string").isin(tract_set)].copy()
    return out

def safe_save_parquet(df: pd.DataFrame, path: Path):
    out = df.copy()

    for c in out.columns:
        if out[c].dtype == "object":
            out[c] = out[c].astype("string")

    out.to_parquet(path, index=False, engine="pyarrow")

def load_igs_any(path: str) -> pd.DataFrame:
    p = Path(path)

    if p.suffix.lower() in [".xlsx", ".xls"]:
        raw = pd.read_excel(p, header=None)
    else:
        raw = pd.read_csv(p, header=None, low_memory=False)

    hdr_i = _find_header_row(raw, required_tokens=("Census Tract FIPS", "Year"))
    header = dedupe_columns(raw.iloc[hdr_i].tolist())

    df = raw.iloc[hdr_i + 1 :].copy()
    df.columns = header
    df = df.dropna(how="all").reset_index(drop=True)

    # Drop any duplicated header rows that still appear in the body
    if "Census Tract FIPS code" in df.columns:
        df = df[
            df["Census Tract FIPS code"].astype(str).str.strip().str.lower()
            .copy()

    # Normalize GEOID + year
    df["geoid"] = normalize_geoid_series(df["Census Tract FIPS code"])
    df["year"] = pd.to_numeric(df["Year"], errors="coerce")

    # Convert non-ID columns to numeric where possible
    for c in df.columns:
        if c not in IGS_STRING_COLS:
            df[c] = pd.to_numeric(df[c], errors="coerce")

    # Friendly aliases for easier modeling later
    for old_col, new_col in IGS_ALIAS_MAP.items():
        if old_col in df.columns:
            df[new_col] = pd.to_numeric(df[old_col], errors="coerce")

    if "Is an Opportunity Zone" in df.columns:
        df["is_opp_zone"] = pd.to_numeric(df["Is an Opportunity Zone"], errors="coerce")

    if "URBAN CODE" in df.columns:

```

```

df["urban_code"] = pd.to_numeric(df["URBAN CODE"], errors="coerce")

if "BENCHMARK" in df.columns:
    df["benchmark"] = df["BENCHMARK"].astype("string")

# Final cleanup
df = df.dropna(subset=["geoid", "year"]).copy()
df["year"] = df["year"].astype(int)
df = df.sort_values(["geoid", "year"]).reset_index(drop=True)

return df

```

Cell 3 — Load and save clean IGS

```

In [16]: igs = load_igs_any(IGS_PATH)
igs = filter_tracts(igs, TARGET_TRACTS)

if SAVE_INTERMEDIATE:
    safe_save_parquet(igs, OUT_DIR / "igs_clean.parquet")

print("IGS shape:", igs.shape)
print("IGS years:", sorted(igs["year"].dropna().unique().tolist())[:10], "...")
print()
print(igs[["geoid", "year", "igs_total"]].head())
print()
print(igs[["igs_total", "igs_place", "igs_economy", "igs_community"]].describe())
print()
print("Top missingness:")
print(igs.isna().mean().sort_values(ascending=False).head(20))

```

```
IGS shape: (765288, 87)
IGS years: [2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025] ... [2023, 2024, 2025]
```

```

      geoid  year  igs_total
0  01001020100  2017      47.0
1  01001020100  2018      52.0
2  01001020100  2019      46.0
3  01001020100  2020      46.0
4  01001020100  2021      38.0

```

	igs_total	igs_place	igs_economy	igs_community
count	757582.000000	761912.000000	764294.000000	762407.000000
mean	50.052242	49.848921	50.328149	50.148915
std	10.252555	12.453479	13.532944	13.744805
min	0.000000	0.000000	0.000000	0.000000
25%	44.000000	42.000000	42.000000	40.000000
50%	50.000000	50.000000	51.500000	50.000000
75%	57.900000	58.000000	60.000000	60.000000
max	84.000000	100.000000	94.000000	100.000000

```

Top missingness:
Is an Opportunity Zone                1.000000
Spend Growth Base, %                  1.000000
is_opp_zone                           1.000000
Spending per Capita Base, %           1.000000
Spending per Capita Tract, %          1.000000
Spend Growth Tract, %                 1.000000
Early Education Enrollment Tract, %   0.176266
Early Education Enrollment Score       0.176266
Minority/Women Owned Businesses Base, % 0.159723
Minority/Women Owned Businesses Score  0.159723
Minority/Women Owned Businesses Tract, % 0.159723
Spend Growth Score                     0.099624
Spending per Capita Score              0.099624
Residential Real Estate Value Score    0.050105
Residential Real Estate Value Tract, % 0.050105
Travel Time to Work Tract, %          0.024643
Travel Time to Work Score             0.024643
Affordable Housing Score               0.019497
Affordable Housing Tract, %           0.019497
Female Above Poverty Tract, %         0.018882
dtype: float64

```

Cell 4 — ACS variables to pull

```

In [17]: ACS_VAR_GROUPS = {
    "population_age": [
        "B01001_001E", # total pop
        "B01001_003E", "B01001_004E", "B01001_005E", "B01001_006E", # male
        "B01001_027E", "B01001_028E", "B01001_029E", "B01001_030E", # female
        "B01001_020E", "B01001_021E", "B01001_022E", "B01001_023E", "B01001_
        "B01001_044E", "B01001_045E", "B01001_046E", "B01001_047E", "B01001_
    ],
    "race": [

```

```

        "B02001_001E",
        "B02001_002E",
        "B02001_003E",
        "B02001_005E",
        "B02001_008E",
    ],
    "education": [
        "B15003_001E",
        "B15003_022E", "B15003_023E", "B15003_024E", "B15003_025E",
    ],
    "employment": [
        "B23025_001E",
        "B23025_002E",
        "B23025_003E",
        "B23025_005E",
    ],
    "income_poverty": [
        "B19013_001E", # median household income
        "B19301_001E", # per-capita income
        "B19083_001E", # gini
        "B17001_001E", # poverty universe
        "B17001_002E", # below poverty
    ],
    "commute": [
        "B08303_001E",
        "B08303_002E", "B08303_003E", "B08303_004E", "B08303_005E",
        "B08303_006E", "B08303_007E", "B08303_008E",
    ],
    "internet": [
        "B28002_001E",
        "B28002_002E",
    ],
    "housing": [
        "B25002_001E", "B25002_002E", "B25002_003E", # total / occupied /
        "B25003_001E", "B25003_002E", "B25003_003E", # tenure
        "B25064_001E", # median gross rent
        "B25077_001E", # median home value
    ],
    "rent_burden": [
        "B25070_001E",
        "B25070_002E", "B25070_003E", "B25070_004E", "B25070_005E", "B25070_
    ],
    "owner_cost_burden": [
        "B25091_001E",
        "B25091_003E", "B25091_004E", "B25091_005E", "B25091_006E", "B25091_
        "B25091_014E", "B25091_015E", "B25091_016E", "B25091_017E", "B25091_
    ],
    "early_education": [
        "B14003_004E", "B14003_013E", "B14003_032E", "B14003_041E",
    ],
    "health_insurance": [
        "B27001_001E",
        "B27001_005E", "B27001_008E", "B27001_011E", "B27001_014E", "B27001_
        "B27001_020E", "B27001_023E", "B27001_026E", "B27001_029E",
        "B27001_033E", "B27001_036E", "B27001_039E", "B27001_042E", "B27001_
        "B27001_048E", "B27001_051E", "B27001_054E", "B27001_057E",
    ],

```

```

    ],
}

ACS_VARS = sorted({v for group in ACS_VAR_GROUPS.values() for v in group})
print("ACS variable count:", len(ACS_VARS))

```

ACS variable count: 98

Cell 5 — ACS API helpers

```

In [18]: def chunks(lst, n):
    for i in range(0, len(lst), n):
        yield lst[i : i + n]

def infer_state_fips_from_tracts(tracts):
    if not tracts:
        return None
    return sorted({str(t).zfill(11)[:2] for t in tracts})

def resolve_acs_years(igs_years, max_available_year=ACS_MAX_AVAILABLE_YEAR):
    igs_years = sorted({int(y) for y in igs_years if pd.notna(y)})
    acs_years = [y for y in igs_years if y <= max_available_year]
    return acs_years

def get_state_fips(year: int, api_key: str) -> list[str]:
    url = f"https://api.census.gov/data/{year}/acs/acs5"
    params = {"get": "NAME", "for": "state:*", "key": api_key}

    r = requests.get(url, params=params, timeout=60)
    r.raise_for_status()

    data = r.json()
    header, rows = data[0], data[1:]
    df = pd.DataFrame(rows, columns=header)

    return sorted(df["state"].unique().tolist())

def census_get_tracts(year: int, state_fips: str, vars_: list[str], api_key: str):
    base = f"https://api.census.gov/data/{year}/acs/acs5"
    get_str = "NAME," + ",".join(vars_)

    params = [
        ("get", get_str),
        ("for", "tract:*"),
        ("in", f"state:{state_fips}"),
        ("in", "county:*"),
        ("key", api_key),
    ]

    r = requests.get(base, params=params, timeout=120)
    r.raise_for_status()

```

```

data = r.json()
header, rows = data[0], data[1:]
df = pd.DataFrame(rows, columns=header)
df["year"] = year

return df

def to_numeric_safe(df: pd.DataFrame, exclude=("NAME", "state", "county", "t
out = df.copy()
for c in out.columns:
    if c in exclude:
        continue
    out[c] = pd.to_numeric(out[c], errors="coerce")
return out

ACS_SENTINELS = {-666666666, -333333333, -222222222}

def clean_acs_values(df: pd.DataFrame, exclude=("NAME", "state", "county", "
out = df.copy()
for c in out.columns:
    if c in exclude:
        continue
    out[c] = pd.to_numeric(out[c], errors="coerce")
    out[c] = out[c].replace(list(ACS_SENTINELS), np.nan)
return out

def download_acs_all(
    years: list[int],
    api_key: str,
    state_fips_filter: list[str] | None = None,
    max_vars_per_call: int = 45,
    sleep_seconds: float = 0.25,
) -> pd.DataFrame:
    if not api_key:
        raise ValueError("Missing CENSUS_API_KEY. Put it in your environment

    all_parts = []

    for year in years:
        year_out = RAW_DIR / f"acs5_tract_{year}.parquet"

        if year_out.exists():
            print(f"[cache] {year_out.name}")
            df_year = pd.read_parquet(year_out)
            all_parts.append(df_year)
            continue

        states = state_fips_filter if state_fips_filter else get_state_fips(
            print(f"[download] year={year} | states={len(states)}")

        state_frames = []

        for st in states:

```



```

merged_state = None

for var_chunk in chunks(ACS_VARS, max_vars_per_call):
    df_chunk = census_get_tracts(year, st, var_chunk, api_key)
    df_chunk = clean_acs_values(df_chunk)

    keys = ["state", "county", "tract", "NAME", "year"]
    if merged_state is None:
        merged_state = df_chunk
    else:
        merged_state = merged_state.merge(df_chunk, on=keys, how='outer')

    time.sleep(sleep_seconds)

state_frames.append(merged_state)

df_year = pd.concat(state_frames, ignore_index=True)

if SAVE_INTERMEDIATE:
    safe_save_parquet(df_year, year_out)

all_parts.append(df_year)

acs = pd.concat(all_parts, ignore_index=True)
return acs

```

Cell 6 — ACS feature engineering

```

In [19]: def safe_div(n, d):
    n = np.asarray(n, dtype="float64")
    d = np.asarray(d, dtype="float64")

    out = np.full_like(n, np.nan, dtype="float64")
    valid = (~np.isnan(d)) & (d != 0)
    np.divide(n, d, out=out, where=valid)
    return out

def make_features(acs: pd.DataFrame) -> pd.DataFrame:
    acs = acs.copy()

    acs["state"] = acs["state"].astype(str).str.zfill(2)
    acs["county"] = acs["county"].astype(str).str.zfill(3)
    acs["tract"] = acs["tract"].astype(str).str.zfill(6)
    acs["geoid"] = acs["state"] + acs["county"] + acs["tract"]

    out = pd.DataFrame({
        "geoid": acs["geoid"],
        "year": acs["year"],
    })

    # -----
    # Population / age
    # -----
    out["pop_total"] = acs["B01001_001E"]

```

```

out["pop_under18"] = (
    acs["B01001_003E"] + acs["B01001_004E"] + acs["B01001_005E"] + acs["
    acs["B01001_027E"] + acs["B01001_028E"] + acs["B01001_029E"] + acs["
)
out["share_under18"] = safe_div(out["pop_under18"], out["pop_total"])

out["pop_65plus"] = (
    acs["B01001_020E"] + acs["B01001_021E"] + acs["B01001_022E"] + acs["
    acs["B01001_044E"] + acs["B01001_045E"] + acs["B01001_046E"] + acs["
)
out["share_65plus"] = safe_div(out["pop_65plus"], out["pop_total"])

# -----
# Race shares
# -----
out["share_white"] = safe_div(acs["B02001_002E"], acs["B02001_001E"])
out["share_black"] = safe_div(acs["B02001_003E"], acs["B02001_001E"])
out["share_asian"] = safe_div(acs["B02001_005E"], acs["B02001_001E"])
out["share_two_plus"] = safe_div(acs["B02001_008E"], acs["B02001_001E"])

# -----
# Education
# -----
ba_plus = acs["B15003_022E"] + acs["B15003_023E"] + acs["B15003_024E"] +
out["ba_plus_share_25p"] = safe_div(ba_plus, acs["B15003_001E"])

# -----
# Employment / economy
# -----
out["lfpr_16p"] = safe_div(acs["B23025_002E"], acs["B23025_001E"])
out["unemp_rate"] = safe_div(acs["B23025_005E"], acs["B23025_003E"])

out["median_household_income"] = acs["B19013_001E"]
out["per_capita_income"] = acs["B19301_001E"]
out["gini"] = acs["B19083_001E"]
out["poverty_rate"] = safe_div(acs["B17001_002E"], acs["B17001_001E"])

# -----
# Commute / internet
# -----
commute_under35 = (
    acs["B08303_002E"] + acs["B08303_003E"] + acs["B08303_004E"] + acs["
    acs["B08303_006E"] + acs["B08303_007E"] + acs["B08303_008E"]
)
out["commute_under35_share"] = safe_div(commute_under35, acs["B08303_001E"])
out["internet_sub_share"] = safe_div(acs["B28002_002E"], acs["B28002_001E"])

# -----
# Housing / affordability
# -----
out["housing_units_total"] = acs["B25002_001E"]
out["occupied_units"] = acs["B25002_002E"]
out["vacant_units"] = acs["B25002_003E"]
out["occupied_share"] = safe_div(acs["B25002_002E"], acs["B25002_001E"])
out["vacancy_rate"] = safe_div(acs["B25002_003E"], acs["B25002_001E"])

```

```

out["owner_share"] = safe_div(acs["B25003_002E"], acs["B25003_001E"])
out["renter_share"] = safe_div(acs["B25003_003E"], acs["B25003_001E"])

out["median_gross_rent"] = acs["B25064_001E"]
out["median_home_value"] = acs["B25077_001E"]

rent_affordable = (
    acs["B25070_002E"] + acs["B25070_003E"] + acs["B25070_004E"] +
    acs["B25070_005E"] + acs["B25070_006E"]
)

owner_affordable = (
    acs["B25091_003E"] + acs["B25091_004E"] + acs["B25091_005E"] + acs["
    acs["B25091_014E"] + acs["B25091_015E"] + acs["B25091_016E"] + acs["
)

denom_housing_cost = acs["B25070_001E"] + acs["B25091_001E"]
out["affordable_housing_share"] = safe_div(rent_affordable + owner_affor

# -----
# Community / health
# -----
# Early education proxy: enrolled 3-4 year olds over under-5 population
enrolled_3_4 = acs["B14003_004E"] + acs["B14003_013E"] + acs["B14003_032
pop_under5 = acs["B01001_003E"] + acs["B01001_027E"]
out["early_ed_enroll_share"] = safe_div(enrolled_3_4, pop_under5)

uninsured = (
    acs["B27001_005E"] + acs["B27001_008E"] + acs["B27001_011E"] + acs["
    acs["B27001_020E"] + acs["B27001_023E"] + acs["B27001_026E"] + acs["
    acs["B27001_033E"] + acs["B27001_036E"] + acs["B27001_039E"] + acs["
    acs["B27001_048E"] + acs["B27001_051E"] + acs["B27001_054E"] + acs["
)
out["insured_share"] = 1.0 - safe_div(uninsured, acs["B27001_001E"])

# -----
# Growth features
# -----
out = out.sort_values(["geoid", "year"]).reset_index(drop=True)

growth_cols = [
    "median_household_income",
    "per_capita_income",
    "occupied_units",
    "median_home_value",
    "median_gross_rent",
]

for col in growth_cols:
    out[f"{col}_growth"] = out.groupby("geoid")[col].pct_change(fill_met

return out

```

Cell 7 — Merge helpers

```
In [20]: def prepare_acs_raw_for_merge(acs: pd.DataFrame) -> pd.DataFrame:
    acs_raw = acs.copy()

    acs_raw["state"] = acs_raw["state"].astype(str).str.zfill(2)
    acs_raw["county"] = acs_raw["county"].astype(str).str.zfill(3)
    acs_raw["tract"] = acs_raw["tract"].astype(str).str.zfill(6)
    acs_raw["geoid"] = acs_raw["state"] + acs_raw["county"] + acs_raw["tract"]

    # one row per tract-year expected
    acs_raw = acs_raw.drop_duplicates(subset=["geoid", "year"]).copy()

    return acs_raw

def merge_igs_acs_raw(igs: pd.DataFrame, acs_raw: pd.DataFrame) -> pd.DataFrame:
    merged = igs.merge(acs_raw, on=["geoid", "year"], how="left", validate="m:m")
    return merged

def merge_igs_acs_full(
    igs: pd.DataFrame,
    acs_raw: pd.DataFrame,
    feats: pd.DataFrame,
) -> pd.DataFrame:
    merged = (
        igs
        .merge(acs_raw, on=["geoid", "year"], how="left", validate="m:m")
        .merge(feats, on=["geoid", "year"], how="left", validate="m:m")
    )
    return merged
```

```
In [21]: # for year in range(2017, 2025):
#         p = RAW_DIR / f"acs5_tract_{year}.parquet"
#         if p.exists():
#             p.unlink()
#             print("deleted", p.name)
```

Cell 8 — Run ACS pull, feature engineering, and merge

```
In [22]: igs_years = sorted(igs["year"].dropna().unique().tolist())
acs_years = resolve_acs_years(igs_years, max_available_year=ACS_MAX_AVAILABLE_YEAR)
state_fips_filter = infer_state_fips_from_tracts(TARGET_TRACTS)

print("IGS years in file:", igs_years)
print("ACS years to pull:", acs_years)
print("State filter from target tracts:", state_fips_filter)

acs = download_acs_all(
    years=acs_years,
    api_key=CENSUS_API_KEY,
    state_fips_filter=state_fips_filter,
    max_vars_per_call=45,
    sleep_seconds=0.25,
```

```

)

acs_raw = prepare_acs_raw_for_merge(acs)
feats = make_features(acs)
feats = filter_tracts(feats, TARGET_TRACTS)

# keep only IGS years that exist in ACS-derived tables
valid_years = sorted(feats["year"].dropna().unique())
model_igs = igs[igs["year"].isin(valid_years)].copy()

igs_x_acs_raw = merge_igs_acs_raw(model_igs, acs_raw)
model_df_full = merge_igs_acs_full(model_igs, acs_raw, feats)

if SAVE_INTERMEDIATE:
    safe_save_parquet(acs, OUT_DIR / "acs_raw.parquet")
    safe_save_parquet(acs_raw, OUT_DIR / "acs_raw_with_geoid.parquet")
    safe_save_parquet(feats, OUT_DIR / "acs_features.parquet")
    safe_save_parquet(igs_x_acs_raw, OUT_DIR / "igs_x_acs_raw.parquet")
    safe_save_parquet(model_df_full, OUT_DIR / "igs_x_acs_full.parquet")

print("ACS raw shape:", acs.shape)
print("ACS raw+geoid shape:", acs_raw.shape)
print("ACS features shape:", feats.shape)
print("IGS x ACS raw shape:", igs_x_acs_raw.shape)
print("Full merged model_df_full shape:", model_df_full.shape)

```

IGS years in file: [2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025]

ACS years to pull: [2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024]

State filter from target tracts: None

[cache] acs5_tract_2017.parquet

[cache] acs5_tract_2018.parquet

[cache] acs5_tract_2019.parquet

[cache] acs5_tract_2020.parquet

[cache] acs5_tract_2021.parquet

[cache] acs5_tract_2022.parquet

[cache] acs5_tract_2023.parquet

[cache] acs5_tract_2024.parquet

ACS raw shape: (648952, 103)

ACS raw+geoid shape: (648952, 104)

ACS features shape: (648952, 37)

IGS x ACS raw shape: (680256, 189)

Full merged model_df_full shape: (680256, 224)

validation cell right after Cell 8

```

In [23]: # choose the final EDA panel
eda_df = model_df_full.copy()

print("EDA shape:", eda_df.shape)
print("EDA years:", sorted(eda_df["year"].dropna().unique().tolist()))
print("EDA unique tracts:", eda_df["geoid"].nunique())

# required ACS raw columns
required_acs_raw = ["NAME", "state", "county", "tract"] + ACS_VARS
missing_acs_raw = [c for c in required_acs_raw if c not in eda_df.columns]

```

```

# required engineered columns
required_feats = [c for c in feats.columns if c not in ["geoid", "year"]]
missing_feats = [c for c in required_feats if c not in eda_df.columns]

# all original IGS columns
missing_igs = [c for c in igs.columns if c not in eda_df.columns]

print("Missing raw ACS columns:", missing_acs_raw[:20], "count =", len(missing_acs_raw))
print("Missing engineered feature columns:", missing_feats[:20], "count =", len(missing_feats))
print("Missing IGS columns:", missing_igs[:20], "count =", len(missing_igs))

print("Duplicate geoid-year rows:", eda_df.duplicated(subset=["geoid", "year"]).sum())

# quick sample
cols_to_show = [
    "geoid", "year", "igs_total", "igs_place", "igs_economy", "igs_community",
    "NAME", "state", "county", "tract",
    "B19013_001E", "B19301_001E", "B25077_001E",
    "median_household_income", "per_capita_income", "median_home_value"
]
print(eda_df[cols_to_show].head())

```

EDA shape: (680256, 224)

EDA years: [2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024]

EDA unique tracts: 85032

Missing raw ACS columns: [] count = 0

Missing engineered feature columns: [] count = 0

Missing IGS columns: [] count = 0

Duplicate geoid-year rows: 0

	geoid	year	igs_total	igs_place	igs_economy	igs_community	
NAME	state	county	tract	B19013_001E	B19301_001E	B25077_001E	\
0	01001020100	2017	47.0	53.0	34.0	54.0	Cens
us Tract 201, Autauga County, Alabama				01	001	020100	67826.0
33018.0			152500.0				
1	01001020100	2018	52.0	48.0	48.0	62.0	Cens
us Tract 201, Autauga County, Alabama				01	001	020100	58625.0
31580.0			133300.0				
2	01001020100	2019	46.0	40.0	40.0	56.0	Cens
us Tract 201, Autauga County, Alabama				01	001	020100	60208.0
31225.0			136100.0				
3	01001020100	2020	46.0	34.0	48.0	56.0	Cens
us Tract 201, Autauga County, Alabama				01	001	020100	60388.0
30447.0			151200.0				
4	01001020100	2021	38.0	43.0	32.0	40.0	Cens
us Tract 201, Autauga County, Alabama				01	001	020100	57399.0
30934.0			138700.0				

	median_household_income	per_capita_income	median_home_value
0	67826.0	33018.0	152500.0
1	58625.0	31580.0	133300.0
2	60208.0	31225.0	136100.0
3	60388.0	30447.0	151200.0
4	57399.0	30934.0	138700.0

Cell 9 — Final checks before EDA

```
In [24]: print("Merged years:", sorted(model_df_full["year"].dropna().unique().tolist))
print()

print(model_df_full[[
    "geoid", "year", "igs_total", "igs_place", "igs_economy", "igs_community",
    "affordable_housing_share", "internet_sub_share", "insured_share",
    "median_household_income", "poverty_rate", "vacancy_rate"
]].head())

print()
print("Top missingness in merged data:")
print(model_df_full.isna().mean().sort_values(ascending=False).head(30))

print()
print("Core numeric summary:")
core_cols = [
    "igs_total",
    "igs_place",
    "igs_economy",
    "igs_community",
    "affordable_housing_share",
    "internet_sub_share",
    "insured_share",
    "median_household_income",
    "per_capita_income",
    "poverty_rate",
    "vacancy_rate",
    "unemp_rate",
]
print(model_df_full[core_cols].describe())
```

Merged years: [2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024]

	geoid	year	igs_total	igs_place	igs_economy	igs_community	affordable_housing_share	internet_sub_share	insured_share	median_household_income	poverty_rate	vacancy_rate
0	01001020100	2017	47.0	53.0	34.0	54.0	0.737401	0.762599	0.909485	67826.0	0.106775	0.014379
1	01001020100	2018	52.0	48.0	48.0	62.0	0.704575	0.717647	0.907436	58625.0	0.113365	0.017972
2	01001020100	2019	46.0	40.0	40.0	56.0	0.715092	0.740480	0.902659	60208.0	0.166583	0.022069
3	01001020100	2020	46.0	34.0	48.0	56.0	0.738817	0.810967	0.903658	60388.0	0.136528	0.023944
4	01001020100	2021	38.0	43.0	32.0	40.0	0.743842	0.840722	0.909548	57399.0	0.153546	0.057276

Top missingness in merged data:

Is an Opportunity Zone	1.000000
Spending per Capita Tract, %	1.000000
Spend Growth Base, %	1.000000
is_opp_zone	1.000000
Spending per Capita Base, %	1.000000
Spend Growth Tract, %	1.000000
median_gross_rent_growth	0.276211
median_home_value_growth	0.256606
median_household_income_growth	0.241214
occupied_units_growth	0.236046
per_capita_income_growth	0.234822
Minority/Women Owned Businesses Tract, %	0.159347
Minority/Women Owned Businesses Base, %	0.159347
Minority/Women Owned Businesses Score	0.159347
Early Education Enrollment Tract, %	0.157891
Early Education Enrollment Score	0.157891
B25064_001E	0.147337
median_gross_rent	0.147337
B25077_001E	0.131784
median_home_value	0.131784
early_ed_enroll_share	0.124228
B19013_001E	0.116411
median_household_income	0.116411
gini	0.113279
B19083_001E	0.113279
affordable_housing_share	0.112346
internet_sub_share	0.112336
owner_share	0.112336
renter_share	0.112336
occupied_share	0.112055
dtype: float64	

Core numeric summary:

igs_total	igs_place	igs_economy	igs_community	affordable_housing_share
-----------	-----------	-------------	---------------	--------------------------

e_housing_share	internet_sub_share	insured_share	median_household_income	per_capita_income	poverty_rate \
count	674227.000000	677454.000000	679363.000000	677817.000000	
	603832.000000	603839.000000	604557.000000	601067.000000	
	605097.000000	604202.000000			
mean	50.092197	49.971285	50.313671	50.150358	
	0.654425	0.847756	0.910665	73893.783232	3
	7332.819006	0.145232			
std	10.325957	12.498407	13.613307	13.857690	
	0.129589	0.115830	0.073407	37973.270136	2
	0408.309777	0.120057			
min	0.000000	0.000000	0.000000	0.000000	
	0.000000	0.000000	0.000000	2499.000000	
	0.000000	0.000000			
25%	44.000000	42.000000	42.000000	40.000000	
	0.575375	0.789683	0.879251	47967.000000	2
	4444.000000	0.059760			
50%	50.100000	50.000000	51.300000	50.000000	
	0.673483	0.873674	0.930211	65578.000000	3
	2765.000000	0.111340			
75%	58.000000	58.000000	60.000000	60.000000	
	0.750096	0.933931	0.962620	90962.000000	4
	4589.000000	0.195118			
max	84.000000	100.000000	94.000000	100.000000	
	1.000000	1.000000	1.000000	250001.000000	69
	1205.000000	1.000000			

	vacancy_rate	unemp_rate
count	604030.000000	604505.000000
mean	0.111957	0.060183
std	0.107473	0.049349
min	0.000000	0.000000
25%	0.042771	0.028459
50%	0.082713	0.048499
75%	0.145941	0.077549
max	1.000000	1.000000

Cell 10 — Optional: save an EDA subset for only chosen tracts

```
In [25]: # Cell 10 – final EDA panel cleanup + save

eda_df = model_df_full.copy() # use model_df_full, not model_df

# drop junk unnamed / NaN column names from the original IGS export
junk_cols = [c for c in eda_df.columns if pd.isna(c) or str(c).startswith("")]
print("Dropping junk columns:", junk_cols)

eda_df = eda_df.drop(columns=junk_cols, errors="ignore")

# optional: reorder key columns to the front
front_cols = [
    c for c in [
        "geoid", "year", "Census Tract FIPS code", "County", "State",
        "igs_total", "igs_place", "igs_economy", "igs_community"
    ]
```

```

    ] if c in eda_df.columns
]
other_cols = [c for c in eda_df.columns if c not in front_cols]
eda_df = eda_df[front_cols + other_cols]

print("EDA panel shape:", eda_df.shape)

if SAVE_INTERMEDIATE:
    safe_save_parquet(eda_df, OUT_DIR / "eda_panel_clean.parquet")

eda_df.head()

```

Dropping junk columns: []
EDA panel shape: (680256, 224)

Out[25]:

	geoid	year	Census Tract FIPS code	County	State	igs_total	igs_place	igs
0	01001020100	2017	01001020100	Autauga County	Alabama	47.0	53.0	
1	01001020100	2018	01001020100	Autauga County	Alabama	52.0	48.0	
2	01001020100	2019	01001020100	Autauga County	Alabama	46.0	40.0	
3	01001020100	2020	01001020100	Autauga County	Alabama	46.0	34.0	
4	01001020100	2021	01001020100	Autauga County	Alabama	38.0	43.0	

5 rows × 224 columns

Cell 11 — validation checks

In [26]: *# Cell 11 – validation checks*

```
print("Duplicate geoid-year rows:", eda_df.duplicated(subset=["geoid", "year"])).sum()

required_acs_raw = ["NAME", "state", "county", "tract"] + ACS_VARS
missing_acs_raw = [c for c in required_acs_raw if c not in eda_df.columns]

required_feats = [c for c in feats.columns if c not in ["geoid", "year"]]
missing_feats = [c for c in required_feats if c not in eda_df.columns]

missing_igs = [c for c in igs.columns if c not in eda_df.columns]

print("Missing raw ACS columns:", len(missing_acs_raw))
print("Missing engineered ACS columns:", len(missing_feats))
print("Missing IGS columns:", len(missing_igs))
```

Duplicate geoid-year rows: 0
Missing raw ACS columns: 0
Missing engineered ACS columns: 0
Missing IGS columns: 0

Cell 12 — shortlist settings + required columns

In [27]: *# Cell 12 – shortlist settings + required columns*

```
SHORTLIST_YEAR = 2024
LOW_IGS_THRESHOLD = 45
PERSIST_YEARS = [2022, 2023, 2024]

TOP_N_NATIONAL = 150
TOP_N_PER_STATE = 3
MIN_POP_FLAG = 1500 # flag only, do not drop automatically

shortlist_required_cols = [
    "geoid", "year",
    "NAME", "state", "county", "tract",
    "County", "State",
    "igs_total", "igs_place", "igs_economy", "igs_community",
    "pop_total", "share_under18", "share_65plus",
    "lfpr_16p", "unemp_rate",
    "median_household_income", "poverty_rate",
    "internet_sub_share", "vacancy_rate",
    "affordable_housing_share", "insured_share",
]

missing_shortlist_cols = [c for c in shortlist_required_cols if c not in eda_df.columns]
print("Missing shortlist columns:", missing_shortlist_cols)

if missing_shortlist_cols:
    raise ValueError(f"Missing required shortlist columns: {missing_shortlist_cols}")

print("Shortlist year:", SHORTLIST_YEAR)
print("Low-IGS threshold:", LOW_IGS_THRESHOLD)
print("Persistence years:", PERSIST_YEARS)
```

Missing shortlist columns: []
Shortlist year: 2024
Low-IGS threshold: 45
Persistence years: [2022, 2023, 2024]

Cell 13 — scoring helpers

```
In [28]: # Cell 13 – scoring helpers

def weighted_percentile_points(series: pd.Series, weight: float, higher_is_worse: bool) -> float:
    """
    Convert a column into weighted percentile-rank points.
    - higher_is_worse=True -> larger values get more points
    - higher_is_worse=False -> smaller values get more points
    """
    ranked = series.rank(
        method="average",
        pct=True,
        ascending=higher_is_worse
    )
    return ranked * weight

def add_score(df: pd.DataFrame, col: str, weight: float, higher_is_worse: bool) -> pd.DataFrame:
    df[out_col] = weighted_percentile_points(df[col], weight=weight, higher_is_worse=higher_is_worse)
    return df
```

Cell 14 — build 2024 base + persistence table

```
In [29]: # Cell 14 – build 2024 base + persistence table

base_2024 = eda_df.loc[eda_df["year"] == SHORTLIST_YEAR].copy()

# display fields: prefer human-readable IGS fields, then fall back to ACS fields
base_2024["display_state"] = base_2024["State"].fillna(base_2024["state"])
base_2024["display_county"] = base_2024["County"].fillna(base_2024["county"])
base_2024["display_name"] = base_2024["NAME"]

persistence = (
    eda_df.loc[eda_df["year"].isin(PERSIST_YEARS), ["geoid", "year", "igs_total"]]
    .dropna(subset=["igs_total"])
    .assign(igs_below_45=lambda d: d["igs_total"] < LOW_IGS_THRESHOLD)
    .groupby("geoid", as_index=False)
    .agg(
        n_persist_years=("igs_below_45", "sum"),
        mean_igs_persist_window=("igs_total", "mean"),
        min_igs_persist_window=("igs_total", "min"),
    )
)

persistence["persistence_score"] = persistence["n_persist_years"].clip(lower=1, upper=3)

shortlist_base = (
    base_2024
    .merge(persistence, on="geoid", how="left")
)
```

```

        .copy()
    )

    shortlist_base["n_persist_years"] = shortlist_base["n_persist_years"].fillna(0)
    shortlist_base["persistence_score"] = shortlist_base["persistence_score"].fillna(0)

    # main eligibility rule
    shortlist_base = shortlist_base.loc[shortlist_base["igs_total"] < LOW_IGS_THRESHOLD]

    # tiny tract flag only
    shortlist_base["tiny_tract_flag"] = shortlist_base["pop_total"] < MIN_POP_FLAG

    print("2024 tracts total:", len(base_2024))
    print("Eligible low-IGS tracts:", len(shortlist_base))
    print("Unique states in shortlist:", shortlist_base["display_state"].nunique())
    print()

    print(shortlist_base[
        [
            "geoid", "display_state", "display_county", "display_name",
            "igs_total", "igs_place", "igs_economy", "igs_community",
            "n_persist_years", "persistence_score"
        ]
    ].head())

```

2024 tracts total: 85032
 Eligible low-IGS tracts: 24329
 Unique states in shortlist: 52

	geoid	display_state	display_county	display_name	igs_total	igs_place	igs_economy	igs_community	n_persist_years	persistence_score
0	01001020100	Alabama	Autauga County	Census Tract 201; Autauga County; Alabama	40.0	43.0	38.0	40.0	3	3.0
10	01001020803	Alabama	Autauga County	Census Tract 208.03; Autauga County; Alabama	42.0	30.0	56.0	39.0	1	1.0
11	01001020804	Alabama	Autauga County	Census Tract 208.04; Autauga County; Alabama	42.0	30.0	56.0	39.0	1	1.0
12	01001020805	Alabama	Autauga County	Census Tract 208.05; Autauga County; Alabama	42.0	30.0	56.0	39.0	1	1.0
13	01001020901	Alabama	Autauga County	Census Tract 209.01; Autauga County; Alabama	42.0	34.0	44.0	51.0	2	2.0

Cell 15 — apply the exact shortlist scorecard

In [30]: *# Cell 15 – apply the exact shortlist scorecard*

```

score_input_cols = [
    "igs_total", "igs_place", "igs_economy", "igs_community",
    "poverty_rate", "insured_share", "median_household_income",

```

```

    "unemp_rate", "affordable_housing_share", "internet_sub_share",
    "share_65plus", "share_under18", "lfpr_16p", "vacancy_rate"
]

tract_shortlist_scored = shortlist_base.dropna(subset=score_input_cols).copy

# -----
# A. IGS distress (45 total)
# -----
tract_shortlist_scored = add_score(tract_shortlist_scored, "igs_total", 20,
tract_shortlist_scored = add_score(tract_shortlist_scored, "igs_economy", 10,
tract_shortlist_scored = add_score(tract_shortlist_scored, "igs_place", 8,
tract_shortlist_scored = add_score(tract_shortlist_scored, "igs_community",

tract_shortlist_scored["igs_distress_score"] = (
    tract_shortlist_scored["igs_total_pts"] +
    tract_shortlist_scored["igs_economy_pts"] +
    tract_shortlist_scored["igs_place_pts"] +
    tract_shortlist_scored["igs_community_pts"] +
    tract_shortlist_scored["persistence_score"]
)

# -----
# B. Healthcare proxy pressure (35 total)
# -----
tract_shortlist_scored = add_score(tract_shortlist_scored, "poverty_rate", 10,
tract_shortlist_scored = add_score(tract_shortlist_scored, "insured_share", 10,
tract_shortlist_scored = add_score(tract_shortlist_scored, "median_household_income", 10,
tract_shortlist_scored = add_score(tract_shortlist_scored, "unemp_rate", 5,
tract_shortlist_scored = add_score(tract_shortlist_scored, "affordable_housing_share", 5,
tract_shortlist_scored = add_score(tract_shortlist_scored, "internet_sub_share", 5,

tract_shortlist_scored["healthcare_proxy_score"] = (
    tract_shortlist_scored["poverty_rate_pts"] +
    tract_shortlist_scored["insured_share_pts"] +
    tract_shortlist_scored["median_household_income_pts"] +
    tract_shortlist_scored["unemp_rate_pts"] +
    tract_shortlist_scored["affordable_housing_share_pts"] +
    tract_shortlist_scored["internet_sub_share_pts"]
)

# -----
# C. Vulnerability / need (20 total)
# -----
tract_shortlist_scored = add_score(tract_shortlist_scored, "share_65plus", 6,
tract_shortlist_scored = add_score(tract_shortlist_scored, "share_under18", 6,
tract_shortlist_scored = add_score(tract_shortlist_scored, "lfpr_16p", 5,
tract_shortlist_scored = add_score(tract_shortlist_scored, "vacancy_rate", 5,

tract_shortlist_scored["vulnerability_score"] = (
    tract_shortlist_scored["share_65plus_pts"] +
    tract_shortlist_scored["share_under18_pts"] +
    tract_shortlist_scored["lfpr_16p_pts"] +
    tract_shortlist_scored["vacancy_rate_pts"]
)

```

```

# final total
tract_shortlist_scored["total_shortlist_score"] = (
    tract_shortlist_scored["igs_distress_score"] +
    tract_shortlist_scored["healthcare_proxy_score"] +
    tract_shortlist_scored["vulnerability_score"]
)

# keep a clean ranking
tract_shortlist_scored = tract_shortlist_scored.sort_values(
    ["total_shortlist_score", "igs_total"],
    ascending=[False, True]
).reset_index(drop=True)

print("Scored shortlist rows:", len(tract_shortlist_scored))
print()

print(tract_shortlist_scored[
    [
        "geoid", "display_state", "display_county", "display_name",
        "igs_total", "igs_place", "igs_economy", "igs_community",
        "igs_distress_score", "healthcare_proxy_score", "vulnerability_score",
        "total_shortlist_score"
    ]
].head(10))

```

Scored shortlist rows: 23873

	geoid	display_state	display_county	display_name	igs_total	igs_place	igs_economy	igs_community	igs_distress
0	47157011500	Tennessee	Shelby County	Census Tract 115; Shelby County; Tennessee	25.0	25.0	28.0	23.0	42.363716
1	72127003300	Puerto Rico	San Juan Municipio	Census Tract 33; San Juan Municipio; Puerto Rico	22.0	32.0	21.0	14.0	42.547313
2	47157005900	Tennessee	Shelby County	Census Tract 59; Shelby County; Tennessee	22.0	28.0	30.0	8.0	41.972731
3	72127003600	Puerto Rico	San Juan Municipio	Census Tract 36; San Juan Municipio; Puerto Rico	25.1	32.8	18.6	23.7	42.073933
4	04017940012	Arizona	Navajo County	Census Tract 9400.12; Navajo County; Arizona	24.0	33.0	21.0	16.0	42.128932
5	47157011200	Tennessee	Shelby County	Census Tract 112; Shelby County; Tennessee	22.0	20.0	23.0	20.0	43.787752
6	46071941200	South Dakota	Jackson County	Census Tract 9412; Jackson County; South Dakota	18.0	20.0	22.0	8.0	44.121853
7	29510110100	Missouri	St. Louis city	Census Tract 1101; St. Louis city; Missouri	22.0	30.0	12.0	24.0	43.235454
8	04017942300	Arizona	Navajo County	Census Tract 9423; Navajo County; Arizona	24.0	24.0	26.0	23.0	42.893562
9	12095010400	Florida	Orange County	Census Tract 104; Orange County; Florida	24.0	24.0	22.0	26.0	43.250576

	vulnerability_score	total_shortlist_score
0	16.708373	91.759100
1	15.601391	91.089515
2	16.331630	90.629875
3	14.168726	90.228291
4	16.845348	90.182591
5	14.536380	89.833159
6	12.717296	89.370272
7	12.220123	89.335442
8	16.103422	89.226532
9	14.437942	88.857286

Cell 16 — build the main shortlist outputs

```
In [31]: # Cell 16 – build the main shortlist outputs
```

```
shortlist_output_cols = [  
    "geoid", "year",  
    "display_state", "display_county", "display_name",
```



```

        "State", "County", "state", "county", "tract",
        "igs_total", "igs_place", "igs_economy", "igs_community",
        "n_persist_years", "mean_igs_persist_window", "min_igs_persist_window",
        "pop_total", "share_under18", "share_65plus",
        "lfpr_16p", "unemp_rate", "median_household_income", "poverty_rate",
        "internet_sub_share", "vacancy_rate", "affordable_housing_share", "insur
        "tiny_tract_flag",
        "igs_distress_score", "healthcare_proxy_score", "vulnerability_score",
        "total_shortlist_score"
    ]

    tract_shortlist_scored = tract_shortlist_scored[shortlist_output_cols].copy()

    top_national_candidates = tract_shortlist_scored.head(TOP_N_NATIONAL).copy()

    state_balanced_candidates = (
        tract_shortlist_scored
        .sort_values(["display_state", "total_shortlist_score"], ascending=False)
        .groupby("display_state", group_keys=False)
        .head(TOP_N_PER_STATE)
        .reset_index(drop=True)
    )

    print("Top national candidates:", top_national_candidates.shape)
    print("State-balanced candidates:", state_balanced_candidates.shape)
    print()

    print(top_national_candidates[
        ["geoid", "display_state", "display_county", "igs_total", "total_shortli
    ].head(20))

    print()
    print(state_balanced_candidates[
        ["display_state", "geoid", "display_county", "igs_total", "total_shortli
    ].head(30))

```

Top national candidates: (150, 33)
 State-balanced candidates: (153, 33)

	geoid	display_state	display_county	igs_total	total_shortli
st_score					
0	47157011500	Tennessee	Shelby County	25.0	9
1.759100					
1	72127003300	Puerto Rico	San Juan Municipio	22.0	9
1.089515					
2	47157005900	Tennessee	Shelby County	22.0	9
0.629875					
3	72127003600	Puerto Rico	San Juan Municipio	25.1	9
0.228291					
4	04017940012	Arizona	Navajo County	24.0	9
0.182591					
5	47157011200	Tennessee	Shelby County	22.0	8
9.833159					
6	46071941200	South Dakota	Jackson County	18.0	8
9.370272					
7	29510110100	Missouri	St. Louis city	22.0	8
9.335442					
8	04017942300	Arizona	Navajo County	24.0	8
9.226532					
9	12095010400	Florida	Orange County	24.0	8
8.857286					
10	22019000200	Louisiana	Calcasieu Parish	24.0	8
8.815189					
11	54069000700	West Virginia	Ohio County	23.0	8
8.797177					
12	29510107200	Missouri	St. Louis city	16.0	8
8.718050					
13	18089011400	Indiana	Lake County	22.0	8
8.707871					
14	01097000500	Alabama	Mobile County	24.0	8
8.467327					
15	17031681200	Illinois	Cook County	19.0	8
8.465442					
16	04001944301	Arizona	Apache County	21.0	8
8.431911					
17	28059041600	Mississippi	Jackson County	26.0	8
8.357894					
18	18089012100	Indiana	Lake County	25.0	8
8.340908					
19	17031530100	Illinois	Cook County	20.0	8
8.031018					

	display_state	geoid	display_county	igs_total
total_shortlist_score				
0	Alabama	01097000500	Mobile County	24.0
88.467327				
1	Alabama	01097001502	Mobile County	20.0
87.668349				
2	Alabama	01097001501	Mobile County	18.0
87.657374				
3	Alaska	02290000100	Yukon-Koyukuk Census Area	32.0
72.989570				

4 67.722511	Alaska	02170000401	Matanuska-Susitna Borough	30.0
5 65.590144	Alaska	02050000300	Bethel Census Area	36.0
6 90.182591	Arizona	04017940012	Navajo County	24.0
7 89.226532	Arizona	04017942300	Navajo County	24.0
8 88.431911	Arizona	04001944301	Apache County	21.0
9 84.442131	Arkansas	05077470100	Lee County	22.1
10 82.394839	Arkansas	05017080200	Chicot County	28.0
11 81.988858	Arkansas	05035031100	Crittenden County	24.0
12 81.865999	California	06025012400	Imperial County	28.0
13 80.098877	California	06037209403	Los Angeles County	32.0
14 79.349726	California	06037530902	Los Angeles County	28.0
15 76.857286	Colorado	08123000600	Weld County	30.0
16 73.474511	Colorado	08123000501	Weld County	32.0
17 68.479412	Colorado	08123001906	Weld County	26.0
18 65.779123	Delaware	10003002200	New Castle County	38.0
19 64.864994	Delaware	10003003002	New Castle County	39.8
20 64.245005	Delaware	10005050703	Sussex County	34.0
21 76.194006	District of Columbia	11001007406	District of Columbia	28.0
22 75.363653	District of Columbia	11001007401	District of Columbia	30.2
23 74.402254	District of Columbia	11001009601	District of Columbia	28.0
24 88.857286	Florida	12095010400	Orange County	24.0
25 87.210824	Florida	12095010500	Orange County	26.0
26 85.219055	Florida	12086002001	Miami-Dade County	29.0
27 87.424350	Georgia	13193000200	Macon County	12.0
28 86.301575	Georgia	13275960100	Thomas County	26.0
29 85.505802	Georgia	13299950400	Ware County	30.0

Cell 17 — county-level candidate community seeds

In [32]: *# Cell 17 – county-level candidate community seeds*

```
candidate_communities = (
    top_national_candidates
    .groupby(["display_state", "display_county"], dropna=False)
    .agg(
        n_shortlisted_tracts=("geoid", "nunique"),
        mean_shortlist_score=("total_shortlist_score", "mean"),
        max_shortlist_score=("total_shortlist_score", "max"),
        mean_igs_total=("igs_total", "mean"),
        mean_poverty_rate=("poverty_rate", "mean"),
        mean_insured_share=("insured_share", "mean"),
        mean_unemp_rate=("unemp_rate", "mean"),
        mean_affordable_housing_share=("affordable_housing_share", "mean"),
        total_pop=("pop_total", "sum"),
        sample_tracts=("geoid", lambda s: ", ".join(s.astype(str).head(5))),
        sample_names=("display_name", lambda s: " | ".join(s.astype(str).head(5)))
    )
    .reset_index()
    .sort_values(
        ["n_shortlisted_tracts", "mean_shortlist_score", "max_shortlist_score"],
        ascending=[False, False, False]
    )
    .reset_index(drop=True)
)

print(candidate_communities.head(25))
```

	display_state	display_county	n_shortlisted_tracts	mean_shortlist_score	max_shortlist_score	mean_igs_total	mean_poverty_rate	mean_insured_share	mean_unemp_rate	\
0	Puerto Rico	San Juan Municipio	14	8	5.792400	91.089515	25.614286	0.607932	0.819181	0.203197
1	Illinois	Cook County	9	8	6.357391	88.465442	22.666667	0.397891	0.849558	0.235932
2	Alabama	Mobile County	8	8	6.916284	88.467327	22.000000	0.438018	0.824336	0.196453
3	Tennessee	Shelby County	7	8	7.296375	91.759100	22.857143	0.478255	0.823091	0.241875
4	Ohio	Cuyahoga County	5	8	6.015930	87.098270	22.520000	0.435460	0.840377	0.122584
5	Michigan	Wayne County	5	8	5.224530	86.572949	21.840000	0.518372	0.911347	0.199342
6	Louisiana	Orleans Parish	5	8	4.754338	87.221820	25.600000	0.430523	0.780681	0.200345
7	Georgia	Bibb County	5	8	4.695112	85.227160	27.400000	0.500733	0.843430	0.201833
8	Indiana	Lake County	4	8	7.046695	88.707871	24.250000	0.491659	0.868145	0.169811
9	Puerto Rico	Arecibo Municipio	4	8	5.504765	87.806225	21.750000	0.645395	0.946609	0.214093
10	Arizona	Navajo County	3	8	8.642874	90.182591	16.000000	0.432138	0.836338	0.164271
11	Missouri	St. Louis city	3	8	7.338988	89.335442	20.000000	0.446765	0.779344	0.250193
12	Arizona	Apache County	3	8	6.701615	88.431911	22.666667	0.414109	0.792306	0.150025
13	Puerto Rico	Guánica Municipio	3	8	4.968318	85.605203	22.666667	0.664305	0.949924	0.265815
14	Florida	Orange County	2	8	8.034055	88.857286	25.000000	0.520890	0.712983	0.160491
15	West Virginia	Ohio County	2	8	7.012158	88.797177	24.800000	0.416770	0.838375	0.183604
16	New Mexico	San Juan County	2	8	5.924402	87.008022	13.000000	0.425332	0.798328	0.120984
17	Alabama	Montgomery County	2	8	4.161982	84.756545	27.000000	0.376847		

0.794372	0.123590		
18	Puerto Rico	Río Grande Municipio	2 8
4.005152	84.659867	22.600000	0.461223
0.892799	0.180524		
19	Texas	Jefferson County	2 8
4.004566	84.166757	27.500000	0.425718
0.775777	0.120761		
20	South Dakota	Jackson County	1 8
9.370272	89.370272	18.000000	0.513158
0.672216	0.117308		
21	Louisiana	Calcasieu Parish	1 8
8.815189	88.815189	24.000000	0.611966
0.895093	0.273810		
22	Mississippi	Jackson County	1 8
8.357894	88.357894	26.000000	0.461949
0.704314	0.132479		
23	Georgia	Macon County	1 8
7.424350	87.424350	12.000000	0.370725
0.836525	0.084034		
24	Texas	Bowie County	1 8
7.404704	87.404704	26.000000	0.540254
0.757384	0.130769		

	mean_affordable_housing_share	total_pop	
sample_tracts \			
0	0.455896	26214.0	72127003300, 72127003600, 72127001400, 72127003701, 72127002800
1	0.343617	13497.0	17031681200, 17031530100, 17031691500, 17031826902, 17031681300
2	0.414796	9149.0	01097000500, 01097001502, 01097001501, 01097004800, 01097001302
3	0.447223	12627.0	47157011500, 47157005900, 47157011200, 47157000600, 47157006000
4	0.391700	6914.0	39035119900, 39035116300, 39035150100, 39035151500, 39035151600
5	0.394520	8033.0	26163530900, 26163530800, 26163515200, 26163533600, 26163514300
6	0.393170	11449.0	22071000903, 22071001401, 22071001746, 22071002700, 22071007501
7	0.444068	11293.0	13021011500, 13021012400, 13021012900, 13021012800, 13021012700
8	0.315927	3030.0	18089011400, 18089012100, 18089011900, 18089010600
9	0.432804	7691.0	72013302201, 72013301600, 72013300500, 72013302300
10	0.646109	6353.0	04017940014, 04017942300, 04017940012, 7940012
11	0.388020	4436.0	29510107200, 29510107400, 2951010100
12	0.689441	11755.0	04001944100, 04001944302, 04001944301, 1944301
13	0.593534	4827.0	72055961200, 72055961300, 72055961500
14	0.174092	2080.0	12095010400, 12095010500
15	0.421457	1592.0	

54069000700, 54069000400		
16	0.677725	5707.0
35045943100, 35045943000		
17	0.452788	5080.0
01101003000, 01101002201		
18	0.608788	6595.0
72119130601, 72119130402		
19	0.424027	2992.0
48245005900, 48245000900		
20	0.495885	1922.0
46071941200		
21	0.450667	1182.0
22019000200		
22	0.431818	2063.0
28059041600		
23	0.504673	1547.0
13193000200		
24	0.296044	1896.0
48037010600		

sample_names

0	Census Tract 33; San Juan Municipio; Puerto Rico Census Tract 36; San Juan Municipio; Puerto Rico Census Tract 1...
1	Census Tract 6812; Cook County; Illinois Census Tract 5301; Cook County; Illinois Census Tract 6915; Cook County...
2	Census Tract 5; Mobile County; Alabama Census Tract 15.02; Mobile County; Alabama Census Tract 15.01; Mobile Cou...
3	Census Tract 115; Shelby County; Tennessee Census Tract 59; Shelby County; Tennessee Census Tract 112; Shelby Co...
4	Census Tract 1199; Cuyahoga County; Ohio Census Tract 1163; Cuyahoga County; Ohio Census Tract 1501; Cuyahoga Co...
5	Census Tract 5309; Wayne County; Michigan Census Tract 5308; Wayne County; Michigan Census Tract 5152; Wayne Cou...
6	Census Tract 9.03; Orleans Parish; Louisiana Census Tract 14.01; Orleans Parish; Louisiana Census Tract 17.46; O...
7	Census Tract 115; Bibb County; Georgia Census Tract 124; Bibb County; Georgia Census Tract 129; Bibb County; Geo...
8	Census Tract 114; Lake County; Indiana Census Tract 121; Lake County; Indiana Census Tract 119; Lake County; Ind...
9	Census Tract 3022.01; Arecibo Municipio; Puerto Rico Census Tract 3016; Arecibo Municipio; Puerto Rico Census Tr...
10	Census Tract 9400.12; Navajo County; Arizona Census Tract 9423; Navajo County; Arizona Census Tract 9400.14; Nav...
11	Census Tract 1101; St. Louis city; Missouri Census Tract 1072; St. Louis city; Missouri Census Tract 1074; St. L...
12	Census Tract 9443.01; Apache County; Arizona Census Tract 9443.02; Apache County; Arizona Census Tract 9441; Apa...
13	Census Tract 9615; Guánica Municipio; Puerto Rico Census Tract 9612; Guánica Municipio; Puerto Rico Census Tract...
14	Census Tract 104; Orange County; Florida Census Tract 105; Orange County; Florida
15	Census Tract 7; Ohio County; West Virginia Census Tract 4; Ohio County; West Virginia
16	Census Tract 9431; San Juan County; New Mexico Census Tract 9430; San Juan County; New Mexico

17 Census Tract 30; Montgomery County; Alabama |
 Census Tract 22.01; Montgomery County; Alabama
 18 Census Tract 1306.01; Río Grande Municipio; Puerto Rico | Census T
 ract 1304.02; Río Grande Municipio; Puerto Rico
 19 Census Tract 59; Jefferson County;
 Texas | Census Tract 9; Jefferson County; Texas
 20
 Census Tract 9412; Jackson County; South Dakota
 21
 Census Tract 2; Calcasieu Parish; Louisiana
 22
 Census Tract 416; Jackson County; Mississippi
 23
 Census Tract 2; Macon County; Georgia
 24
 Census Tract 106; Bowie County; Texas

Cell 18 — save outputs for team discussion

```
In [33]: # Cell 18 – save shortlist outputs for team discussion

if SAVE_INTERMEDIATE:
    safe_save_parquet(tract_shortlist_scored, OUT_DIR / "tract_shortlist_sco
    safe_save_parquet(top_national_candidates, OUT_DIR / "top_national_candi
    safe_save_parquet(state_balanced_candidates, OUT_DIR / "state_balanced_c
    safe_save_parquet(candidate_communities, OUT_DIR / "candidate_communitie

    tract_shortlist_scored.to_csv(OUT_DIR / "tract_shortlist_scored.csv", index=
    top_national_candidates.to_csv(OUT_DIR / "top_national_candidates.csv", inde
    state_balanced_candidates.to_csv(OUT_DIR / "state_balanced_candidates.csv",
    candidate_communities.to_csv(OUT_DIR / "candidate_communities.csv", index=Fa

    print("Saved shortlist outputs to:", OUT_DIR)
```

Saved shortlist outputs to: data\processed

Cell 19 — quick sanity checks before you present results to the team

```
In [34]: # Cell 19 – quick sanity checks before sharing with the team

print("Top 10 states by count in national shortlist:")
print(top_national_candidates["display_state"].value_counts().head(10))

print()
print("Top 10 counties by count in national shortlist:")
print(
    top_national_candidates
    .assign(state_county=lambda d: d["display_county"].astype(str) + ", " +
    ["state_county"]
    .value_counts()
    .head(10)
)
```



```
print()
print("Tiny tract flags in national shortlist:")
print(top_national_candidates["tiny_tract_flag"].value_counts(dropna=False))

print()
print("Summary of final shortlist scores:")
print(top_national_candidates[
    ["igs_distress_score", "healthcare_proxy_score", "vulnerability_score",
    ].describe())
```

Top 10 states by count in national shortlist:

display_state

Puerto Rico	37
Alabama	16
Georgia	11
Illinois	10
Louisiana	9
Tennessee	8
Mississippi	8
Ohio	6
Arizona	6
Michigan	5

Name: count, dtype: int64

Top 10 counties by count in national shortlist:

state_county

San Juan Municipio, Puerto Rico	14
Cook County, Illinois	9
Mobile County, Alabama	8
Shelby County, Tennessee	7
Bibb County, Georgia	5
Wayne County, Michigan	5
Cuyahoga County, Ohio	5
Orleans Parish, Louisiana	5
Arecibo Municipio, Puerto Rico	4
Lake County, Indiana	4

Name: count, dtype: int64

Tiny tract flags in national shortlist:

tiny_tract_flag

False	99
True	51

Name: count, dtype: int64

Summary of final shortlist scores:

	igs_distress_score	healthcare_proxy_score	vulnerability_score	total_score
count	150.000000	150.000000	150.000000	
mean	41.646020	29.483917	14.576241	
std	1.978924	2.407299	1.614854	
min	35.852218	23.188539	10.070163	
25%	40.322865	28.065786	13.509383	
50%	42.023018	29.484491	14.672894	
75%	43.126932	31.468448	15.606292	
max	44.803586	34.020421	19.203493	

In []: